Fluierul Final — plan aplicație SMA

Oct 23, 2026 · @Mihai

Conceptul

Fluierul Final este un Flashscore personal: urmărește în profunzime doar echipele tale favorite, reacționează fizic la ce se întâmplă în meciurile lor și îți explică orice meci cu ajutorul AI.

Ce o face diferită față de o aplicație de scoruri obișnuită:

- **Telefonul simte meciul:** sunet la gol, vibrații cu pattern diferit la roșu și penalty, notificare la echipele de start.
- **Știe unde ești:** te anunță de meciurile din zonă și îți face miercurea un plan de weekend (vineri seara meciul A, sâmbătă la prânz meciul B etc.).
- **Îți explică meciul:** analiză AI pe statistici reale pentru orice meci, inclusiv La Liga sau Premier League, plus chat cu întrebări de continuare.
- **Te implică:** pronosticuri și pariuri cu puncte virtuale la cote reale, cu clasament între prieteni.

Cum bifăm cerințele SMA

Toate cele 10 puncte sunt acoperite cu rezervă: CRUD pe mai multe colecții, 4 componente hardware față de 2 cerute și 8 ecrane față de 3–4.

Cerință	Puncte	Cum o îndeplinim	Rezervă
CRUD (Firebase permis)	5	Echipe favorite cu setări de alertă per echipă (create, read, update, delete)	Pronosticuri, jucători favoriți, plan de weekend editabil, conversații AI salvate
Minim 2 senzori / actuatoare	2	Senzor de luminozitate (temă automată) + GPS (meciuri în zonă)	Vibrație (roșu, penalty) + sunet (gol) ca actuatoare
3–4 ecrane	3	Login, Favorite, Explorează, Detalii meci	Detalii echipă, Plan weekend, Meciuri în zonă, Setări
Milestones	—	S8, S10, S12, cu livrabile clare (vezi planul pe milestones)	Mod de simulare pentru demo

Funcționalități: MVP vs. bonusuri

Regula: MVP-ul aduce nota maximă și se termină până la S10; bonusurile se adaugă în ordinea de mai jos doar după ce MVP-ul merge complet.

MVP (obligatoriu)

- Autentificare cu Firebase Auth (email sau Google).
- Echipe favorite: căutare echipă, adăugare, ștergere, setări de alertă per echipă.
- Ecran Explorează: ligă, apoi dată, apoi lista de meciuri (orice ligă, nu doar favorite).
- Detalii meci: scor, evenimente, statistici.
- Meciuri live ale favoritelor cu detectare de evenimente: sunet la gol, vibrații la roșu și penalty.
- Senzor de luminozitate pentru tema automată.
- GPS: meciuri în raza aleasă.
- Mod de simulare (gol, roșu, penalty, echipe de start) pentru demo.

Bonusuri, în ordinea raport efort / impresie

#	Bonus	Efort	De ce merită
1	Analiză AI a meciului + chat	Mic	Cel mai mare efect la prezentare, un apel Gemini și un prompt bun
2	Notificare la echipele de start	Mic	Aceeași logică de worker ca la evenimentele live
3	Plan de weekend (miercurea)	Mediu	Funcție originală, cu algoritm propriu de prioritizare
4	Cote, probabilități și pariuri virtuale	Mediu	Încă un CRUD și clasament între prieteni
5	Function calling în chat	Mediu	Chat liber: „analizează Barcelona – Sevilla de sâmbătă”
6	Model ML propriu (TFLite on-device)	Mare	Doar dacă rămâne timp; nu aduce puncte în plus

Am scos din plan The Odds API (nu are liga din România) și LLM-ul local antrenat (efort mare, rezultat mai slab decât un model statistic).

Ecranele

Opt ecrane, dintre care primele patru sunt MVP; navigația principală e o bară jos cu Favorite, Explorează, Plan weekend și Setări.

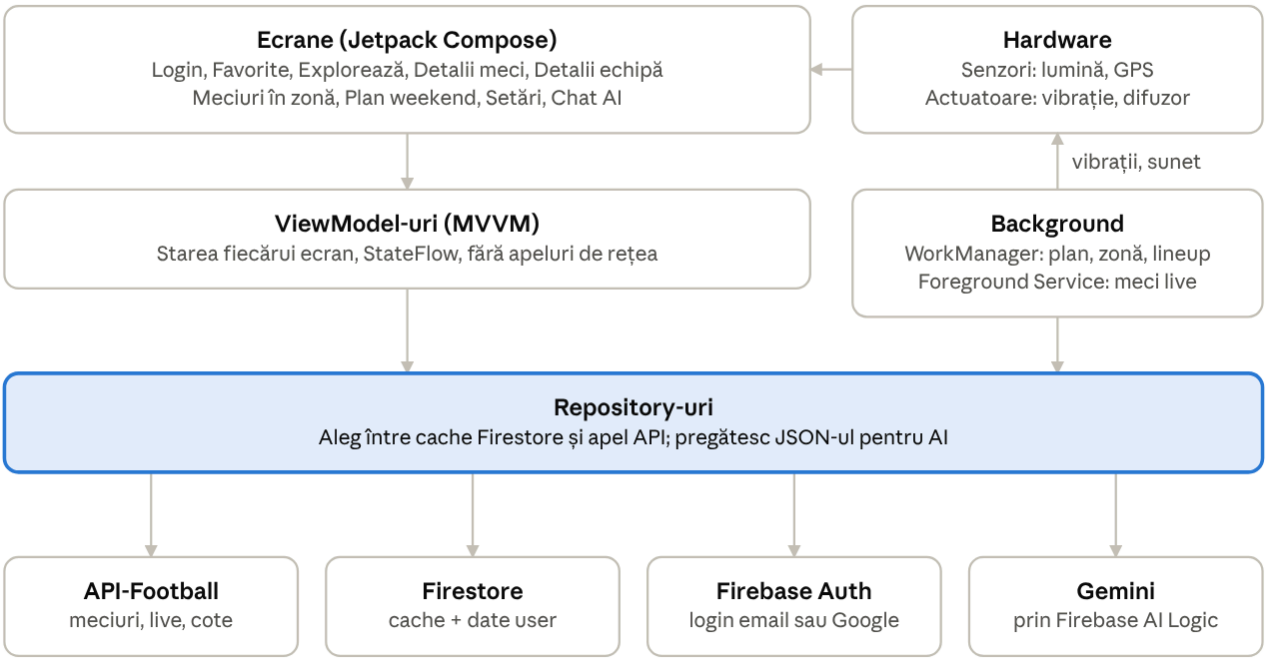
Ecran	Ce conține	Milestone
Login / Register	Firebase Auth, email sau Google	S8
Favorite	Lista echipelor (CRUD), meciurile live primele, forma pe ultimele 5	S8
Explorează	Ligă, apoi dată, apoi meciuri; orice ligă (SuperLiga, La Liga etc.)	S8
Detalii meci	Scor, cronologie evenimente, statistici, echipe de start, butonul „Analiză AI”	S10
Detalii echipă	Clasament, formă, goluri acasă / deplasare, golgheteri, meciuri viitoare	S10
Meciuri în zonă	Listă + hartă cu stadioanele din rază	S10
Plan weekend	Planul generat miercurea, editabil (schimbă / scoate / adaugă meci)	S12
Setări	Temă (auto / light / dark), rază GPS, ziua planului, mod de simulare	S12

Chatul AI se deschide din Detalii meci, nu e ecran separat în bara de navigație.

Arhitectură și tehnologii

Android nativ în Kotlin cu Jetpack Compose și MVVM; tot backend-ul e Firebase, deci nu scriem niciun server.

Ecranele nu vorbesc direct cu API-ul: totul trece prin repository-uri



arhitectura MatchPulse · 4 straturi

Senzorul de lumină și GPS-ul alimentează ecranele (tema, meciurile din zonă), iar serviciile din background declanșează vibrațiile și sunetul. Singurul strat care atinge rețeaua sau Firestore este cel de repository-uri.

Zonă	Tehnologie
UI	Jetpack Compose, Navigation Compose, Material 3, Google Maps Compose
Rețea	Retrofit + kotlinx.serialization, OkHttp cu interceptor pentru cheia API
Backend	Firebase Auth, Cloud Firestore, Firebase AI Logic (Gemini)
Background	WorkManager, Foreground Service, NotificationManager
Hardware	SensorManager, Fused Location Provider, Geocoder, Vibrator, MediaPlayer
Imagini	Coil pentru logo-urile echipelor

Surse de date

O singură sursă externă, API-Football, cu planul gratuit de 100 de request-uri pe zi; Firestore e cache comun pentru toți utilizatorii, deci un meci odată descărcat nu mai costă nimic. Sursă: API-Football – ghid de început.

Endpoint-uri folosite

Nevoie	Endpoint	Cache
Ligi + ce date au (coverage)	`/leagues?country=Romania`, `/leagues?id=140`	O dată pe sezon
Căutare echipă	`/teams?search=Rapid`	Permanent
Meciuri pe zi / ale unei echipe	`/fixtures?league=X&date=...`, `/fixtures?team=X&next=5`	O dată pe zi
Toate meciurile live + evenimente	`/fixtures?live=all` (sau filtrat pe ligi)	Fără cache, polling la 2 min
Meci complet (evenimente, statistici, echipe, jucători)	`/fixtures?id=X`	Permanent după final
Echipe de start	`/fixtures/lineups?fixture=X`	Permanent după ce apar
Clasament / statistici sezon	`/standings`, `/teams/statistics`	O dată pe zi
Head-to-head	`/fixtures/headtohead?h2h=A-B`	O dată pe zi
Cote	`/odds?fixture=X`	O dată pe zi
Stadion (pentru GPS)	`/venues?id=X` + Geocoder Android	Permanent

Buget zilnic estimat (zi cu meciuri)

Consum	Request-uri
Polling live la 2 min, un meci de 90 min	~55
Echipe de start (favorite)	~5
Analize AI pe meciuri noi	~10
Clasamente, meciuri viitoare, cote	~15
Total	**~85 din 100**

Un singur apel `live=all` acoperă toate meciurile live simultan, deci mai multe favorite care joacă în același timp nu cresc consumul. Dacă bugetul nu ajunge la prezentare, planul Pro costă 19 dolari pentru o lună (7.500 de request-uri pe zi).

Cheia API stă în `local.properties` (expusă prin `BuildConfig`) și e în `.gitignore` din prima zi; în producție ar sta într-o Cloud Function.

Model de date Firestore

Două zone: date per utilizator (CRUD-ul notat) și cache comun cu datele din API, citit de toți.

```
users/{uid}
  favorites/{teamId}      → name, logoUrl, leagueId, addedAt,
                            alerts: { goalSound, redCardVibrate,
penaltyVibrate, lineups }
  favoritePlayers/{playerId} → name, teamId
  predictions/{fixtureId} → homeScore, awayScore, note, points, createdAt
  virtualBets/{betId}     → fixtureId, market, pick, odd, stake, status
  weekendPlans/{weekId}   → generatedAt, items: [{ fixtureId, slot, mode:
stadion | tv, confirmed }]
  chats/{fixtureId}/messages/{msgId} → role, text, createdAt
  settings                → theme, gpsRadiusKm, planDay, planHour,
lastLocation

matches/{fixtureId}       → status, score, events, statistics, lineups,
players, updatedAt
analyses/{fixtureId}      → aiSummary, model, createdAt
teams/{teamId}            → name, logo, venueId, seasonStats, updatedAt
venues/{venueId}          → name, city, lat, lng
standings/{leagueId_season} → table, updatedAt
odds/{fixtureId}          → h2h, overUnder, impliedProb, updatedAt
```

Reguli de securitate: fiecare utilizator citește și scrie doar sub `users/{uid}`; colecțiile de cache sunt citibile de oricine autentificat. Regula de cache: meciul terminat se păstrează permanent, meciul live se reîmprospătează.

Senzori și actuatoare

Patru componente hardware: doi senzori (luminozitate, GPS) și două actuatoare (vibrație, difuzor). Cerința cere două, deci rămân două de rezervă.

Componentă	Tip	Ce face în aplicație	API Android
Luminozitate	Senzor	Sub ~25 lux trece pe dark, peste ~60 lux pe light (histerezis, ca să nu clipească)	`SensorManager`, `Sensor.TYPE_LIGHT`
GPS	Senzor	Meciuri în raza aleasă, sugestii „adaugă la favorite”, plan de weekend	`FusedLocationProviderClient`, `Geocoder`, `Location.distanceBetween`
Vibrație	Actuator	Pattern diferit pe eveniment, recunoscut fără să te uiți la telefon	`Vibrator`, `VibrationEffect.createWaveform`
Difuzor	Actuator	Sunet de gol pentru echipa favorită, alt sunet la gol primit	`MediaPlayer` + fișiere în `res/raw`

Pattern-uri de vibrație propuse

Eveniment	Pattern
Gol	Sunet + o vibrație scurtă
Cartonaș roșu	O vibrație lungă (~800 ms)
Penalty	Scurt, scurt, lung
Echipe de start	Două vibrații scurte

GPS-ul se citește doar cu aplicația deschisă și ultima locație se salvează în `settings.lastLocation`; worker-ele din background folosesc locația salvată, ca să evităm permisiunea specială de locație în background.

Notificări și background

Totul rulează pe telefon, fără server: WorkManager pentru sarcinile programate și un Foreground Service doar cât timp o favorită joacă live.

Notificare	Declanșare	Mecanism
Gol / roșu / penalty	Eveniment nou în `live=all` față de apelul anterior	Foreground Service cu polling la 2 min, pornit la începutul meciului, oprit la final
Echipe de start	Lineup-ul devine disponibil (de obicei cu 30–60 min înainte)	`OneTimeWorkRequest` la start − 70 min, se reprogramează la 10 min până apare lineup-ul
Meci în zonă	Meci în rază, cu câteva ore înainte	Worker zilnic dimineața, pe locația salvată
Plan de weekend	Miercurea la ora aleasă	Worker periodic zilnic care verifică ziua; notificare `InboxStyle` cu meciurile pe sloturi
Rezumat AI după meci	Meciul unei favorite s-a terminat	Același serviciu, la status FT: analiză Gemini + notificare

Algoritmul planului de weekend

1. Ia meciurile de vineri până duminică: cele din zonă și cele ale favoritelor (cele departe sunt marcate „la TV”).
2. Le împarte pe sloturi: vineri seara, sâmbătă prânz, sâmbătă seara, duminică prânz, duminică seara.
3. Pentru fiecare slot alege meciul cu scorul cel mai mare: favorită +100, apropiere (invers proporțional cu distanța), meci echilibrat după cote, meci de top în clasament.
4. Elimină suprapunerile: un meci ține ~2 ore, plus drumul dacă e la stadion.
5. Salvează planul în `weekendPlans/{weekId}` și trimite notificarea; tap deschide ecranul Plan weekend.

Android poate întârzia worker-ele câteva minute (Doze mode); pentru aplicația noastră e acceptabil.

AI: analiză, chat și cote

Chatbotul nu știe nimic despre meciuri: aplicația îi aduce statisticile din API (prin cache) și i le pune în prompt, iar el doar le explică. Modelul este Gemini prin Firebase AI Logic, care merge pe planul gratuit Spark, fără server (documentație).

Fluxul analizei post-meci

1. Utilizatorul deschide orice meci (favorite sau din Explorează) și apasă „Analiză AI”.

2. Aplicația ia `matches/{fixtureId}` din cache sau face un singur apel `/fixtures?id=X`.
3. Calculează în Kotlin indicatorii: precizie la șut, conversie, xG dacă există, indice de dominare, momentele cheie, omul meciului.
4. Trimite JSON-ul compact la Gemini cu promptul de mai jos; răspunsul se salvează în `analyses/{fixtureId}`.
5. Utilizatorul continuă cu întrebări în același chat („de ce a pierdut posesia în repriza a doua?").

Structura răspunsului (din system prompt)

1. Verdict: cine a jucat mai bine și dacă rezultatul e meritat.
2. Cum a jucat fiecare echipă: stil, puncte tari, slăbiciuni.
3. Momentele cheie.
4. Omul meciului și de ce.
5. Detaliul surprinzător: o statistică ce contrazice scorul.

Reguli: doar datele primite, cifre concrete, nimic inventat; spune clar când datele lipsesc.

Alte funcții AI

- Preview înainte de meci: formă, clasament, head-to-head, cote.
- Comentariu la echipele de start: „3 schimbări față de etapa trecută, trece pe 3-5-2".
- Analiză pe cote: probabilitatea implicită (1/cotă, normalizată) comparată cu statisticile; exprimată doar în probabilități, fără „pariu sigur" și fără recomandări de mize.
- Function calling (bonus): modelul cere singur `findFixture`, `getMatchStats`, `getTeamLastMatches`, `getStandings`, iar aplicația execută apelurile.

Pariurile sunt doar cu puncte virtuale (1.000 pe săptămână), fără linkuri către case de pariuri, cu confirmare 18+ pentru ecranul de cote.

Biletul meu și programul zilei

Utilizatorul își introduce meciurile de pe bilet, iar aplicația îi face programul de urmărire pe ore și îi arată live starea fiecărui pariu. Este bonusul cu cel mai mult sens pentru cine pune bilete: vrea să știe când începe fiecare meci și cum stă biletul, fără să caute prin aplicații.

Exemplu de program generat

Ora	Meci	Pariu	Stare
14:00	Poli Timișoara – U Cluj	1	Câștigat
16:00	Rapid – FCSB	Peste 2.5	Live: 2 goluri, mai trebuie unul

Ora	Meci	Pariu	Stare
16:30	Girona – Betis	GG	În paralel cu Rapid – FCSB
19:00	Barcelona – Sevilla	1	Începe în 2h 10min

Cum funcționează

1. Utilizatorul creează un bilet: adaugă meciuri din Explorează sau le caută după nume, iar la fiecare alege pariul (1 / X / 2, peste / sub 2.5, GG / NG, rezultat la pauză).
2. Aplicația sortează meciurile după ora de start și le pune pe o cronologie verticală. Un meci ține ~1h50 cu pauza; dacă două se suprapun, sunt marcate „în paralel”.
3. La meciurile în paralel: urmărești unul, iar de la celălalt primești doar alertele importante (gol, roșu, pariu decis).
4. Cu 10 minute înainte de fiecare meci vine o notificare; la finalul unui meci, notificarea spune ce urmează: „Urmează Barcelona – Sevilla la 19:00”.
5. Starea fiecărui pariu se calculează din scorul live (același apel `live=all`, fără request-uri în plus): în desfășurare, câștigat sau pierdut. Când un pariu se decide, telefonul vibrează scurt; la final, starea întregului bilet.

Detalii de implementare

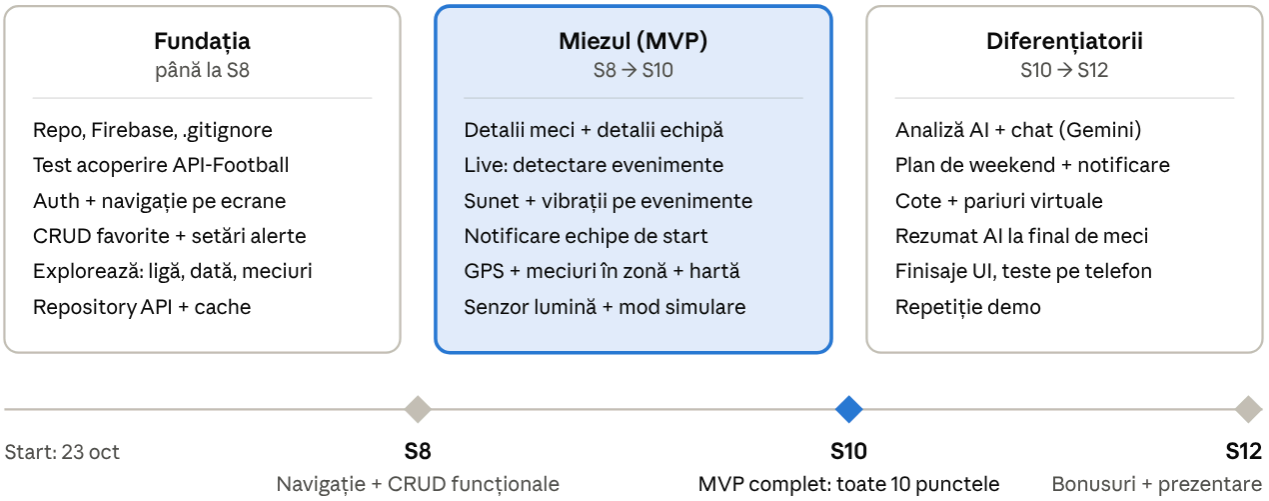
- CRUD nou: `users/{uid}/tickets/{ticketId}` cu `selections: [{ fixtureId, kickoff, market, pick, status }]`; biletul se poate edita până începe primul meci și se poate șterge oricând.
- Evaluarea pariurilor e o funcție pură în Kotlin (`evaluate(market, pick, score, status)`), ușor de testat cu unit tests.
- Pe cronologie, meciul live e evidențiat, iar cel următor arată un countdown.
- Fără sume, fără linkuri către case de pariuri: aplicația urmărește biletul, nu îl plasează.
- Analiza AI poate comenta biletul la final: „Ai pierdut doar la GG, Betis n-a marcat deși a avut 9 șuturi”.

Efort: mediu. Intră în S12, după analiza AI, pentru că refolosește polling-ul live și notificările din S10.

Plan pe milestones

MVP-ul (toate cele 10 puncte) trebuie să meargă complet la S10; S12 e rezervat bonusurilor care diferențiază aplicația și repetiției de demo.

MVP-ul complet la S10, bonusurile doar după



plan pe milestones · 3 faze, 3 porți

Dacă o fază întârzie, se taie din bonusurile din faza a treia, niciodată din MVP. Primul lucru de făcut, înainte de orice cod, este testul de acoperire API-Football.

Împărțirea în echipă

Presupunem o echipă de 3; fiecare are o zonă clară, iar modelele de date comune (clasele Kotlin și structura Firestore) se stabilesc împreună în prima săptămână, ca nimeni să nu-l blocheze pe altul.

Rol	Responsabilități	Ecrane
UI și navigație	Compose, temă light / dark, bara de navigație, hartă, liste, grafice pe ecranul echipei	Favorite, Explorează, Detalii meci, Detalii echipă, Plan weekend
Date și Firebase	Auth, Firestore + reguli, repository API-Football, cache, CRUD-uri, buget de request-uri	Login, Setări
Hardware, background și AI	Senzori și actuatoare, WorkManager, Foreground Service, notificări, mod de simulare, Gemini	Meciuri în zonă, chat AI

- [ ] Repo comun pe GitHub, cu `.gitignore` pentru chei înainte de primul commit
- [ ] Proiect Firebase comun, toți trei adăugați ca membri
- [ ] Cont API-Football + testul de acoperire (vezi Riscuri)

- [ ] Modelele de date comune scrise și aprobate

Riscuri și verificări

Cel mai mare risc e acoperirea datelor pe planul gratuit; se verifică cu un singur request, înainte de orice linie de cod.

Risc	Cum îl verificăm	Plan de rezervă
Sezonul curent nu e accesibil pe planul gratuit	`/leagues?country=Romania` și `/leagues?id=140`: câmpul `coverage` și sezoanele	Planul Pro o lună (19 $) sau demo pe sezonul precedent + simulare
Lipsesc evenimente, statistici sau cote pentru SuperLiga / Liga 2	`coverage.fixtures.events`, `statistics_fixtures`, `odds`	Favoritele de demo din La Liga; cotele ascunse pentru ligile fără acoperire
Bugetul de 100 request-uri / zi se termină	Contorul din headerul `x-ratelimit-requests-remaining`, afișat în Setări (debug)	Polling la 3–5 min, cache mai agresiv
Nu e niciun meci live la prezentare	—	Modul de simulare
Scope prea mare pentru timpul disponibil	Bifăm MVP-ul la S10 înainte de orice bonus	Bonusurile 4–6 rămân în afara proiectului
Cheia API urcată pe GitHub	`.gitignore` din prima zi	Regenerare cheie din dashboard
Coordonate lipsă pentru stadioane	Geocoding pe oraș o singură dată	Colecția `venues` completată manual pentru ligile din demo

Scenariul de demo

Prezentarea durează ~5 minute și arată fiecare punct notat în ordinea baremului, apoi bonusurile.

1. **Login** și ecranul Favorite gol.
2. **CRUD:** căutăm și adăugăm Rapid și Barcelona, schimbăm setările de alertă la una, ștergem o echipă.

3. **Senzor de lumină:** acoperim senzorul cu degetul, tema trece pe dark; luăm degetul, revine pe light.
4. **GPS:** ecranul Meciuri în zonă arată meciurile din raza aleasă, pe hartă.
5. **Simulare live:** din Setări, „Simulează gol”: sună telefonul; „Simulează roșu”: vibrație lungă; „Simulează penalty”: scurt-scurt-lung.
6. **Ecrane:** Explorează, apoi un meci din La Liga, apoi Detalii meci cu statistici.
7. **Analiză AI** pe acel meci, plus o întrebare de continuare în chat.
8. **Plan de weekend:** „Generează planul acum”, notificarea, apoi editarea unui slot.

Se pregătește dinainte: un meci terminat deja salvat în cache (ca analiza să nu depindă de rețea) și o locație cu meciuri în rază.